CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

The aim of this project was to design and implement an Online Campus Waste Management System (OCWMS) for the University of Cross River State (UNICROSS). The system was developed to address the challenge of poor waste disposal and the lack of a structured, accountable channel through which students, sanitation staff, and campus administrators can report, track, and resolve waste incidents.

Chapter One introduced the problem, outlining the objectives of the system and the scope of the work. Chapter Two reviewed existing literature and related systems, establishing the theoretical foundation for web-based waste management solutions, artificial intelligence in waste classification, and geographic information systems for campus mapping. Chapter Three described the system analysis, including the requirements gathering methodology and the fact-finding techniques adopted. Chapter Four presented the system design and implementation, covering the three-tier architecture, the technology stack, the database structure, the input and output screen formats, the program algorithms, flowcharts, and the hardware and software requirements of the system.

In this chapter, a summary of the complete work is presented, followed by a discussion of the results achieved, the conclusion drawn from the study, recommendations for the university, the limitations encountered, and suggestions for future research.

5.2 Results and Discussion

5.2.1 System Implementation Outcome

The OCWMS was fully implemented and tested as a functional web-based application. The final system consists of a Single-Page Application (SPA) frontend built with React and TypeScript, a backend server built with Node.js and Express.js, and a file-based JSON database for persistence. The system exposes twenty-two (22) RESTful API endpoints

organised across six route modules, in addition to artificial intelligence classification and health-check endpoints.

The implementation achieved all seven design objectives set out in Chapter Three:

Table 5.1: Summary of Achieved Objectives

S/N	Objective	Status	Evidence
1	Centralised waste reporting	Achieved	Students can submit reports with photo, category, priority, and location
2	AI-powered classification	Achieved	Google Gemini API classifies waste images with confidence scores
3	Role-based access control	Achieved	Three distinct portals for Student, Staff, and Administrator
4	Real-time task management	Achieved	Reports move through Pending → Assigned → In Progress → Completed
5	Interactive campus mapping	Achieved	MapLibre GL map showing bins and collection vehicles
6	Automated notifications and audit trail	Achieved	Notifications generated for every action; activity logs maintained
7	Analytics and reporting	Achieved	Admin dashboard with charts, KPIs, and trend data

5.2.2 System Testing Results

The system was subjected to comprehensive testing at the unit, integration, and system levels.

Unit Testing: The backend API was tested against twenty-nine (29) test cases covering all major functions:

Table 5.2: API Test Results

Test Category	Number of Tests	Status
User registration for all three roles	3	Passed
User login and session retrieval	3	Passed
Waste report creation	1	Passed
Report assignment to staff	2	Passed
Report status updates (accept, complete)	2	Passed
Report comments	1	Passed
Notifications (list, single read, read-all)	3	Passed
Activity logs retrieval	1	Passed
AI waste classification	1	Passed
Map data (bins and trucks)	2	Passed
User management (list, update, suspend, reactivate, role change, delete)	6	Passed

Test Category	Number of Tests	Status
Profile update	1	Passed
Total	29	All Passed

All twenty-nine (29) tests passed with no failures, confirming that the backend correctly handles registration, authentication, report lifecycle management, staff assignment, notifications, activity logging, AI classification, map data, and user administration.

Integration Testing: The frontend application was successfully connected to the backend through the REST API client. Every user action in the browser — from login to report submission to task resolution — was verified to persist correctly to the database and reflect across all user roles.

Type Checking: The TypeScript compiler reported zero errors across the entire codebase, confirming type safety in both frontend and backend code.

Production Build: The system was built successfully for production deployment. The build process produced an optimised single-file server bundle and a compressed SPA frontend, ensuring efficient deployment on any cloud platform or local server.

5.2.3 Discussion of Results

The successful completion of all tests confirms that the system is reliable, functional, and ready for deployment. The role-based access control ensures that each category of user interacts only with the features relevant to their responsibilities, reducing complexity and improving usability.

The AI classification feature, which uses the Google Gemini API with a keyword-based heuristic fallback, provides students with immediate guidance on waste categories and correct disposal methods. This directly supports the university's environmental sustainability goals by educating users at the point of reporting.

The real-time task management workflow — from student submission through staff assignment to photographic evidence of completion — introduces an unprecedented level of accountability to campus sanitation operations. Every stage of the process is documented in the activity log, creating a complete audit trail that can be reviewed at any time.

The interactive campus map provides operational visibility to administrators, allowing them to monitor bin fill levels and collection vehicle status in real time. This supports data-driven decisions about collection routes and resource allocation.

5.3 Conclusion

The research successfully demonstrated that a comprehensive waste management system can be designed and implemented using modern web technologies at minimal cost. The Online Campus Waste Management System provides a practical, user-friendly platform that connects students, sanitation staff, and administrators in a single coordinated workflow.

The system fulfils its objectives by enabling students to report waste incidents with photographic evidence and AI-assisted classification; empowering staff with a clear task list and a structured accept-and-resolve workflow; and giving administrators the analytics, user management, and audit capabilities required to run an efficient campus sanitation operation.

The choice of a three-tier architecture with a RESTful API ensures that the system is scalable, maintainable, and extensible. The use of the MapLibre mapping library with free OpenStreetMap tiles eliminates recurring licensing costs, while the JSON-based database provides a simple, zero-configuration storage solution suitable for a prototype. The abstraction layer around the database ensures that a production-grade database management system can be integrated without redesigning the application.

Overall, the system represents a significant improvement over manual waste reporting methods and provides the university with a foundation for building a smarter, more sustainable campus.

5.4 Recommendations

Based on the findings of this project, the following recommendations are made:

1. **Adoption by the University:** The University of Cross River State should adopt the Online Campus Waste Management System as part of its environmental sustainability programme. The system should be deployed on the university's official web server and accessible to all students and staff.
2. **Mobile Application:** The university should commission the development of a mobile application version of the system to give students and staff access from their

smartphones. The existing RESTful API is already compatible with mobile clients and can be reused without modification.

3. **Production Database:** For full-scale deployment, the JSON file database should be replaced with a relational database management system such as PostgreSQL or MongoDB. This will improve performance, security, and data integrity when handling large numbers of concurrent users.
4. **Password Security:** In a production deployment, user passwords should be hashed using a secure algorithm such as bcrypt, and session management should be implemented using JSON Web Tokens (JWT) to strengthen authentication security.
5. **Integration with Smart Bins:** The university should consider procuring sensor-equipped smart bins that feed real-time fill-level data into the system automatically, replacing manual bin data entry.
6. **Training and Awareness:** The university should organise training sessions and awareness campaigns to educate students and staff on how to use the system and the importance of proper waste segregation and disposal.
7. **Deployment on Cloud Infrastructure:** The system should be deployed on a cloud platform such as Railway, Render, or a university-managed server, with automatic scaling and regular data backups to ensure high availability.

5.5 Limitations of the Study

The following limitations were encountered during the course of this project:

1. **Lack of Real Production Data:** The system runs on a prototype database, and the test data represents a limited sample. A full pilot run with real users would provide more accurate performance and usability data.
2. **AI Classification Dependency:** The AI classification feature requires an internet connection and a Google Gemini API key. In the absence of the key, the system falls back to a keyword-based heuristic classifier, which is less accurate for images.

3. **File-Based Database:** The JSON file database is not suitable for handling very large amounts of concurrent data without performance degradation. It is best suited for prototype and small-scale usage.
4. **No Mobile Optimisation:** Although the web interface is responsive, it does not provide the native mobile experience that a dedicated mobile application would offer.
5. **No Real-Time Notifications:** Notifications are generated and displayed in-app on page load. Push notifications to mobile devices are not yet supported.
6. **Internet Dependency:** The map rendering and AI classification features require an internet connection, limiting offline usability.

5.6 Suggestions for Further Research

Future research and development on this system may explore the following areas:

1. **Machine Learning Model Development:** Train a custom convolutional neural network (CNN) on local waste image datasets to eliminate dependency on third-party AI services and improve classification accuracy for Nigerian waste types.
2. **Predictive Route Optimisation:** Develop algorithms that predict waste accumulation patterns and automatically optimise collection vehicle routes, reducing fuel consumption and operational costs.
3. **Real-Time IoT Integration:** Integrate Internet of Things (IoT) sensors on waste bins to automatically transmit fill-level data, turning the system into a fully autonomous smart waste monitoring platform.
4. **Gamification for Students:** Introduce rewards, badges, and leaderboards to encourage student participation in waste reporting and recycling, increasing engagement and environmental awareness.
5. **Integration with University Systems:** Integrate the waste management system with the university's student portal, learning management system, and administrative databases for seamless single-sign-on access.

6. **Waste-to-Value Analytics:** Extend the analytics module to track waste collection volumes over time and connect them with recycling and composting programmes, providing the university with measurable sustainability metrics.

REFERENCES

Boehm, B. (1988). *A Spiral Model of Software Development and Enhancement*. ACM SIGSOFT Software Engineering Notes, 11(4), 14-24.

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.

Google (2026). *Gemini API Documentation*. Retrieved from https://ai.google.dev

MapLibre (2026). *MapLibre GL JS Documentation*. Retrieved from https://maplibre.org

Meta (2026). *React Documentation*. Retrieved from https://react.dev

OpenStreetMap (2026). *OpenStreetMap Wiki*. Retrieved from https://wiki.openstreetmap.org

Pressman, R. S. (2015). *Software Engineering: A Practitioner's Approach* (8th ed.). New York: McGraw-Hill Education.

React (2026). *React 19 Documentation*. Retrieved from https://react.dev

Somerville, I. (2016). *Software Engineering* (10th ed.). London: Pearson Education.

Tailwind Labs (2026). *Tailwind CSS Documentation*. Retrieved from https://tailwindcss.com

UNICROSS (2026). *University of Cross River State Official Website*. Retrieved from https://www.unicross.edu.ng